

Отчет по практическим работам №3.1 и №3.2 Ефремов ЭФБО-10-23

Часть 1: Контейнеризация приложения (Docker Compose)

Цель

Целью первого этапа была контейнеризация трехкомпонентного приложения (frontend, backend, database) с использованием Docker для последующего развертывания.

Dockerfile для Backend

Для backend-сервиса на Node.js применен многоэтапный подход для уменьшения размера финального образа. На первом этапе (`builder`) устанавливаются все зависимости и компилируется TypeScript. На втором, финальном, этапе копируются только скомпилированные JS-файлы и production-зависимости.

```
FROM node:20-slim AS builder

WORKDIR /app

COPY package.json package-lock.json* ./
RUN npm install

COPY . .
RUN npx tsc

FROM node:20-slim

WORKDIR /app

COPY package.json package-lock.json* ./
RUN npm install --production

COPY --from=builder /app/dist ./dist

EXPOSE 3000

CMD ["node", "dist/index.js"]
```

Dockerfile для Frontend

- **Этап `builder`**: Собирает статические файлы приложения с помощью `npm`
`run build`.
- **Финальный этап**: Использует легковесный образ `nginx:stable-alpine` для раздачи собранной статики. Также копируется кастомный `nginx.conf` для корректной работы SPA-роутинга.

```
FROM node:20-slim AS builder

WORKDIR /app

COPY package.json package-lock.json* ./
RUN npm install

COPY . .
RUN npm run build

FROM nginx:stable-alpine

COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Файл `docker-compose.yml`

Этот файл использовался для оркестрации сборки образов. Он определяет три сервиса и связывает их вместе.

```
services:
  db:
    image: postgres:14-alpine
    restart: always
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: defects_db
    volumes:
      - ./db-init/init.sql:/docker-entrypoint-initdb.d/init.sql
      - postgres_data:/var/lib/postgresql/data
    ports:
```

– "5432:5432"

```
backend:
  build: ./backend
  image: devops/backend:latest
  restart: always
  depends_on:
    - db
  environment:
    DATABASE_URL: "postgresql://user:password@db:5432/defects_db"
    PORT: 3000
  ports:
    - "3000:3000"

frontend:
  build: ./frontend
  image: devops/frontend:latest
  restart: always
  ports:
    - "5173:80"

volumes:
  postgres_data:
```

Часть 2: Развертывание в кластере (Docker Swarm + Nginx)

Настройка Reverse Proxy

Для маршрутизации внешних запросов к внутренним сервисам был настроен Nginx. Он слушает порт 80 и перенаправляет запросы:

- URL, начинающиеся с `/api/`, направляются на `backend`.
- Все остальные URL направляются на `frontend`.

Содержимое `nginx/nginx.conf`:

```
events {}
http {
    server {
        listen 80;

        location / {
            proxy_pass http://frontend:80;
```

```

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /api/ {
        proxy_pass http://backend:3000/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
}

```

Настройка SPA-роутинга для Frontend

Чтобы избежать ошибки 404 при прямой загрузке или обновлении страниц React-приложения (например, `/login`), Nginx-сервер, обслуживающий frontend, был настроен всегда возвращать `index.html`.

Содержимое `frontend/nginx.conf`:

```

server {
    listen 80;
    root /usr/share/nginx/html;
    include /etc/nginx/mime.types;

    location / {
        try_files $uri $uri/ /index.html;
    }
}

```

Файл стека `docker-stack.yml`

```

version: '3.8'

services:
  frontend:
    image: devops/frontend:latest
    networks:

```

- app-net

```
backend:
  image: devops/backend:latest
  environment:
    DATABASE_URL: "postgresql://user:password@db:5432/defects_db"
    PORT: 3000
  depends_on:
    - db
  networks:
    - app-net
```

```
db:
  image: postgres:14-alpine
  environment:
    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
    POSTGRES_DB: defects_db
  volumes:
    - ./db-init/init.sql:/docker-entrypoint-initdb.d/init.sql
    - postgres_data:/var/lib/postgresql/data
  networks:
    - app-net
```

```
nginx:
  image: nginx:latest
  ports:
    - "80:80"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf
  depends_on:
    - frontend
    - backend
  networks:
    - app-net
```

```
networks:
  app-net:
    driver: overlay
```

```
volumes:
  postgres_data:
```

1. **Инициализация Swarm:** `docker swarm init`
2. **Сборка образов:** `docker compose build`
3. **Развертывание стека:** `docker stack deploy -c docker-stack.yml app`

```
~/Documents/devops new_branch !? > docker stack services app
ID                NAME           MODE           REPLICAS  IMAGE                                  PORTS
23azoxv47v29     app_backend    replicated      1/1        devops/backend:latest                *
zn5omagfeudp     app_db         replicated      1/1        postgres:14-alpine                   *
8ka1p8f7col7     app_frontend   replicated      1/1        devops/frontend:latest               *
mzbctf3nwe3h     app_nginx      replicated      1/1        nginx:latest                          *:80→80/tcp
```

ИнструментыWorkflowNew TabProblem Dev · sowlcommunitycurl - Dockfrontend x

←→↻http://localhost

СистемаКонтроляРоль: ENGINEERСменить рольВыйти

Панель управления дефектами

Создать дефектВыйти

Название	Описание	Приоритет	Статус	Исполнитель	Действия
asdf	asdf	LOW	NEW	aleksey4094@yandex.ru	Редактировать

Ссылка на репозиторий

<https://github.com/sowwds/devops>